Министерство науки и вышего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования

« Московский педагогический государственный университет »

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №8

Выполнил студент группы ЗОМ34-ПВШ2616: **Золотухин Андрей Александрович**

Ссылка на репозиторий: https://github.com/CorgiPuppy/dslp-labs

Принял: Жданов Сергей Александрович

Дата сдачи: 31.10.26

Москва

2026

СОДЕРЖАНИЕ

1. Задание 1 3
   1.1. Ознакомление с примерами из лекции 3
2. Задание 2 10
   2.1. Решение теста 10
3. Задание 3 11

1. Задание 1

1.1. Ознакомление с примерами из лекции

```
// functor1.cpp
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей
Александрович

/*
 * Функторы – это объекты, которые можно рассматривать
 * как функцию или указатель на функцию.
 * Функторы чаще всего используются вместе с STL в сценарии,
 * подобном следующему:
*/
// C++ программа использует алгоритм transform() из STL
// и добавляет  1 ко всем элементам массива arr[]

#include <iostream>
#include <iomanip>
#include <algorithm>

int increment(int x) { return (x + 1); }

int main() {
  int arr[] = {1, 2, 3, 4, 5}; // массив
  int n = sizeof(arr)/sizeof(arr[0]); // размер массива
  std::cout << "Input array: ";
  for (int i = 0; i < n; i++)
    std::cout << arr[i] << " ";
  std::cout << "\n";

  //Применяем приращение ко всем элементам
  // arr[] и сохраняем измененные элементы
  // назад в arr []
  std::transform(arr, arr+n, arr, increment);

  std::cout << "Transform array by functor: ";
  for (int i = 0; i < n; i++)
    std::cout << arr[i] << " ";
  std::cout << "\n";

  return 0;
}
```

```
Input array: 1 2 3 4 5
Transform array by functor: 2 3 4 5 6
```

```cpp
//functor2.cpp
// C++ программа демонстрирует работу функтора
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей Александрович

/*
 * Функтор (или объект функции) — это класс C++, который действует как функция.
 * Функторы вызываются с использованием того же старого синтаксиса вызова функции.
 * Чтобы создать функтор, мы создаем объект, который перегружает оператор().
*/

#include <iostream>
#include <iomanip>
#include <functional> // функторы
#include <algorithm>

// Объект Функтор
class increment {
  private:
    int num;
  public:
    increment(int n): num(n) { }
    // Эта перегрузка оператора позволяет вызывать оператор function() для объекта increment
    int operator () (int arr_num) const {
      return num + arr_num;
    }
};

// исполняемый код
  int main(){
  int arr[] = {1, 2, 3, 4, 5};
  int n = sizeof(arr)/sizeof(arr[0]);
  std::cout << "Input array: ";
  for (int i = 0; i < n; i++)
    std::cout << arr[i] << " ";
  std::cout<<"\n";

  int to_add = 5;
  std::cout << "Increment: " << to_add << std::endl;

  std::transform(arr, arr+n, arr, increment(to_add)); // добавляет заданное
```

```
число

  std::cout << "Transform array by functor: ";
  for (int i = 0; i < n; i++)
    std::cout << arr[i] << " ";
  std::cout << "\n";

  return 0;
}
```

```
Input array: 1 2 3 4 5
Increment: 5
Transform array by functor: 6 7 8 9 10
```

```
// prlambda1.cpp
// C++ программа для демонстрации лямбда-выражения
// Автор ОЗНАКОМЛЕН и протестировал: студент группы ЗОМ34-ПВШ2616 Золотухин
Андрей Александрович

#include <bits/stdc++.h> // STL

// Функция для печати вектора
void printVector(std::vector<int> v){
  // lambda expression to print vector
  for_each(v.begin(), v.end(), [](int i) {
      std::cout << i << " ";
  });
  std::cout << std::endl;
}

// основная программа
int main(){
  // исходный вектор данных
  std::vector<int> v {4, 1, 3, 5, 2, 3, 1, 7};
  std::cout << "Input array: ";
  printVector(v);

  // фрагмент ниже  находит первое число больше
  // find_if ищет элемент, для которого
  // функция (третий аргумент) возвращает true
  std::vector<int>::iterator p = find_if(v.begin(), v.end(), [](int i) {
    return i > 6;
  });
  std::cout << "First number greater than 6 is: " << *p << std::endl;
```

```
  // функция для сортировки вектора, лямбда-выражение для сортировки в
  // невозрастающем порядке Компилятор может определить возвращаемый тип как
bool, но здесь показано только для пояснения
  std::sort(v.begin(), v.end(), [](const int& a, const int& b) -> bool {
    return a > b;
  });
  std::cout << "Sorted array: ";
  printVector(v);

  // функция для подсчета чисел больше или равных 5
  int count_5 = count_if(v.begin(), v.end(), [](int a) {
    return (a >= 3);
  });
  std::cout << "The number of elements greater than or equal to 3 is: " <<
count_5 << std::endl;

  // функция для удаления дублирующего элемента (после сортировки всех
дубликатов объединяется)
  p = unique(v.begin(), v.end(), [](int a, int b) {
    return a == b;
  });

  // изменение размера вектора, чтобы сделать его равным общему различному
числу
  v.resize(std::distance(v.begin(), p));
  std::cout << "Uniqued array: ";
  printVector(v);

  // функция накопления накапливает контейнер на основе
  // функции, указанной в качестве третьего аргумента
  int arr[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15};
  long long f = std::accumulate(arr, arr + 15, 1LL, [](long long i, int j) {
    return i * j;
  });
  std::cout << "Factorial of 15 is: " << f << std::endl;

  // Мы также можем получить доступ к функции, сохранив ее в переменной
  auto square = [](int i) {
    return i * i;
  };

  std::cout << "Square of 25 is: " << square(25) << std::endl;

  return 0;
}
```

```
Input array: 4 1 3 5 2 3 1 7
First number greater than 6 is: 7
Sorted array: 7 5 4 3 3 2 1 1
The number of elements greater than or equal to 3 is: 5
Uniqued array: 7 5 4 3 2 1
Factorial of 15 is: 1307674368000
Square of 25 is: 625
```

Ubuntu-26.04

```
corgi@LAPTOP-AS2LVH3V:~/git-repositories/dslp-labs/labs/lab8/СДАП - ЛР_8/Задание 1/src$ cat prlambda1.cpp
// prlambda1.cpp
// C++ программа для демонстрации лямбда-выражения
// Автор ОЗНАКОМЛЕН и протестировал: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей Александрович

#include <bits/stdc++.h> // STL

// Функция для печати вектора
void printVector(std::vector<int> v){
  // lambda expression to print vector
  for_each(v.begin(), v.end(), [](int i) {
      std::cout << i << " ";
  });
  std::cout << std::endl;
}

// основная программа
int main(){
  // исходный вектор данных
  std::vector<int> v {4, 1, 3, 5, 2, 3, 1, 7};
  std::cout << "Input array: ";
  printVector(v);

  // фрагмент ниже  находит первое число больше
  // find_if ищет элемент, для которого
  // функция (третий аргумент) возвращает true
  std::vector<int>::iterator p = find_if(v.begin(), v.end(), [](int i) {
    return i > 6;
  });
  std::cout << "First number greater than 6 is: " << *p << std::endl;

  // функция для сортировки вектора, лямбда-выражение для сортировки в
  // невозрастающем порядке Компилятор может определить возвращаемый тип как bool, но здесь показано только для пояснения
  std::sort(v.begin(), v.end(), [](const int& a, const int& b) -> bool {
    return a > b;
  });
  std::cout << "Sorted array: ";
  printVector(v);
```

Рис. 1: Авторская иллюстрация программы prlambda1.cpp

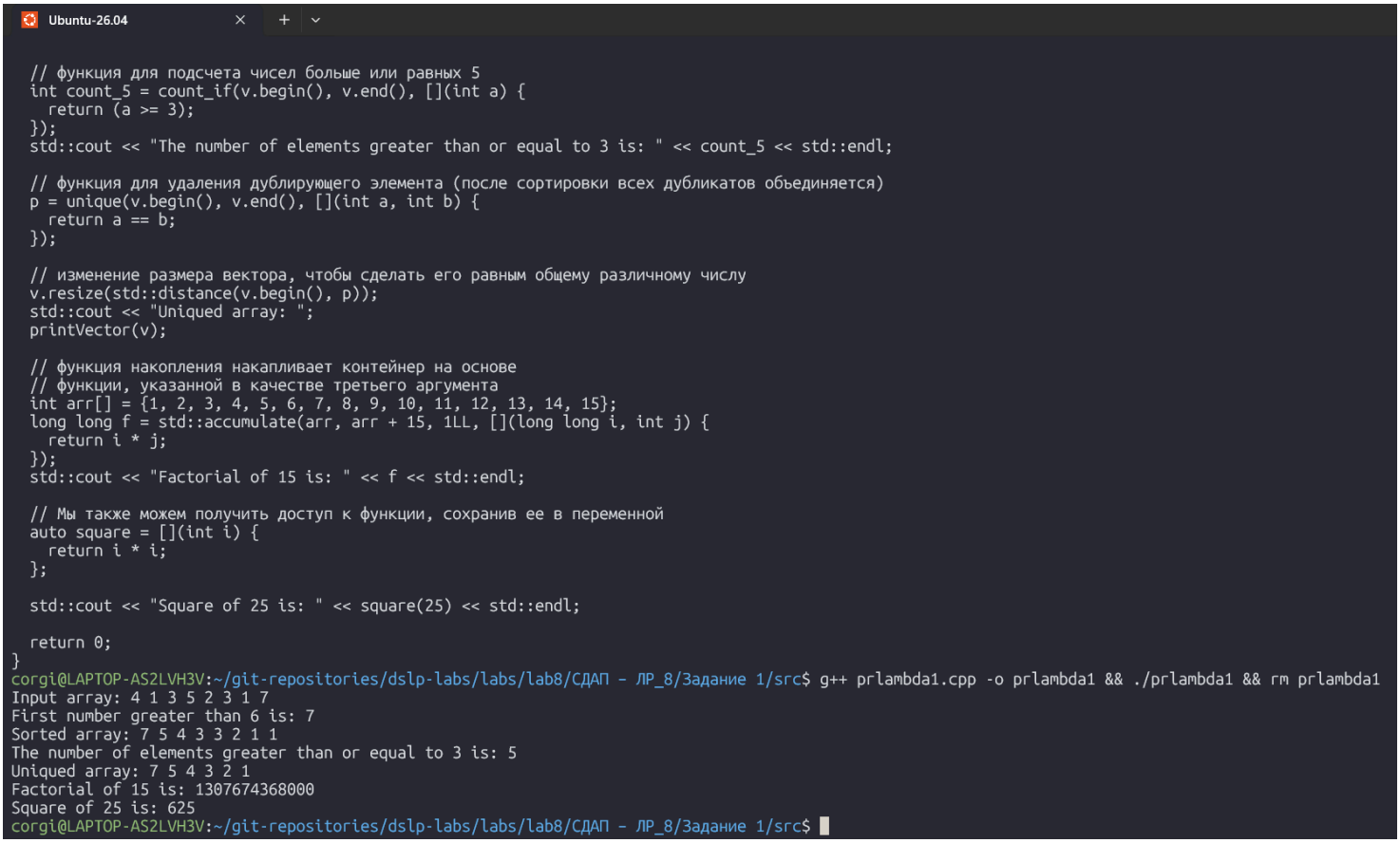


Рис. 2: Авторская иллюстрация программы prlambda1.cpp (продолжение)

```cpp
// prlambda2.cpp
// Лямбда и C++ STL
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей
Александрович

/*
 * Следующий код разобьёт массив из 10-ти натуральных чисел на четные и
нечетные числа.
 */

#include <iostream>
#include <vector>
#include <algorithm>

int main(){
  int n = 100;
  std::vector<int> v;
  for (int i = 0; i < n; i = (i % 2 == 0) ? i + 1 : i * 2)
    // разнообразия добавил другую последовательность
    v.push_back(i);

  std::cout << "Array: ";
  // определим функцию вывода чисел из вектора как лямбда-выражение
```

```
  for_each(v.begin(), v.end(), [] (int val) {
     std::cout << val << " ";
  });
  std::cout << std::endl;

  std::cout << "Even: ";
  // определим функцию выбора четных чисел из вектора как лямбда-выражение
  for_each(v.begin(), v.end(), [] (int val) {
    if (val % 2 == 0) std::cout << val << " ";
  });

  std::cout << std::endl << "Odd: ";
  // определим функцию выбора нечетных чисел из вектора как лямбда-выражение
  for_each( v.begin(), v.end(), [] (int val) {
    if (val % 2 == 1) std::cout << val << " ";
  });
  std::cout << std::endl;

  return 0;
}
```

```
Array: 0 1 2 3 6 7 14 15 30 31 62 63
Even: 0 2 6 14 30 62
Odd: 1 3 7 15 31 63
```

2. Задание 2

2.1. Решение теста

Sat Oct 31 2026 14:52:27 GMT+0300 (Москва, стандартное время)
Дисциплина: Структуры данных, языки и методы программирования (СДАП)
Тема: Лямбда-функция в программировании(ЛР-8)
Форма контроля: Вопросы для самопроверки в режиме диалога
Группа: ЗОМ34-ПВШ2616

Вопросы для самопроверки (с заполнением в режиме диалога)

по теме: **Лямбда-функция в программировании(ЛР-8)**

Введите фамилию и имя
Студент:
Золотухин Андрей
Ok

Вопрос	Ответ
1. Что такое лямбда-функция в Python? Ваш выбор: ◦ Встроенная функция для выполнения математических вычислений ◦ Функция, определенная с помощью ключевого слова "def" ◦ Анонимная функция, определенная с помощью ключевого слова "lambda" ◦ Функция, которая может использоваться только со строками	Анонимная функция, определенная с помощью ключевого слова "lambda"
2. Какой будет результат выполнения следующего кода: x = lambda a, b : a - b print(x(5, 3)) Ваш выбор: ◦ 15 ◦ 8 ◦ 3 ◦ 2	2
3. Верно или неверно утверждение: лямбда-функции могут принимать несколько аргументов. Ваш выбор: ◦ Да ◦ Нет	Да
4. Верно или неверно: лямбда-функции могут иметь несколько выражений. Ваш выбор: ◦ Верно ◦ Неверно	Неверно
5. Создайте лямбда-функцию, которая принимает один параметр (a) и возвращает его. (Укажите код в ответе)	Язык Python: lambda a: a
6. Какой будет результат выполнения следующего кода? nums = [1, 2, 3, 4, 5] result = list(map(lambda x: x * x, nums)) print(result) Ваш выбор: ◦ [1, 2, 3, 4, 5] ◦ [1, 4, 9, 16, 25] ◦ [2, 4, 6, 8, 10] ◦ [0, 1, 2, 3, 4]	[1, 4, 9, 16, 25]
7. Что выведет следующий фрагмент кода? f = lambda x, y: x + y result = f(2, 3) print(result) Ваш выбор: ◦ 2 ◦ 3 ◦ 5 ◦ 6	5
8. Что делает следующая лямбда-функция? f = lambda x: x if x % 2 == 0 else None result = f(5) print(result) Ваш выбор: ◦ Возвращает число, если оно четное, в противном случае — None. ◦ Возвращает None, если число четное, в противном случае — None. ◦ Возвращает квадрат числа, если оно четное, в противном случае — None. ◦ Возвращает число, если оно нечетное, в противном случае — None.	Возвращает число, если оно четное, в противном случае — None.
9. Какой результат выдаст следующий код? func_list = [lambda x: x + 2, lambda x: x * 2, lambda x: x ** 2] results = [func(5) for func in func_list] print(results) Ваш выбор: ◦ [7, 10, 5] ◦ [10, 25, 7] ◦ [7, 10, 25] ◦ [10, 5, 25]	[7, 10, 25]
10. Какой из следующих вариантов является правильным использованием лямбда-функции с несколькими аргументами? Ваш выбор: ◦ lambda x, y: x * y ◦ lambda (x, y): x * y ◦ lambda x y: x * y ◦ lambda x: x * y	lambda x, y: x * y

Текст ответа на вопрос:

Сохранить: 1 2 3 4 5 6 7 8 9 10 – номера вопросов

Отменить: 1 2 3 4 5 6 7 8 9 10 – номера вопросов

Введите/Отмените текст ответа на вопрос и нажмите кнопку с его номером для сохранения или удаления.

© Золотухин Андрей

3. Задание 3

```cpp
/* Автор: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей Александрович
 * СДАП Лабораторная работа 8
 * Язык: C++
 */

#include <iostream>
#include <vector>
#include <algorithm>
#include <iterator>

// Функция для печати вектора
void printVector(std::vector<int> v){
  std::cout << "[";
  // лямбда-выражние для вывода вектора
  int idx = 0; // использую, чтобы считать индекс
  for_each(v.begin(), v.end(), [&](int i) {
    if (idx++ != v.size() - 1) std::cout << i << ", ";
    else std::cout << i; // на последнем элементе не вывожу запятую с пробелом
  });
  std::cout << "]" << std::endl;
}

int main() {
  /*
   * Задача 1
   * возведение чисел в определённую степень (в частности, в квадрат и куб)
   */
  std::cout << "--- Задача 1 ---" << std::endl;
  std::cout << "Возведение чисел в определённую степень (в частности, в
квадрат и куб)" << std::endl;
  int n = 10; // количество элементов массива
  // заполнение элементами массив от 1 до 10
  std::vector<int> v;
  for (int i = 0; i < n; i++)
    v.push_back(i + 1);

  std::cout << "Исходный массив: ";
  printVector(v);

  std::vector<int> squares(n); // создаю отдельный массив для записи в него
элементов исходного, возведённых в квадрат
  std::vector<int> cubes(n); // создаю отдельный массив для записи в него
элементов исходного, возведённых в куб

  // Применяю ко всем элементам вектора v и сохраняю измененные элементы в
```

```
squares
  std::transform(v.begin(), v.end(), squares.begin(), [](int i) {
    return i * i;
  });
  std::cout << "Возведите в квадрат каждое число: ";
  printVector(squares);

  // Применяю ко всем элементам вектора v и сохраняю измененные элементы в
cubes
  std::transform(v.begin(), v.end(), cubes.begin(), [](int i) {
    return i * i * i;
  });
  std::cout << "Возведите в куб каждое число: ";
  printVector(cubes);
  std::cout << std::endl;
  // очистка вектора
  v.clear();
  /*
   * Конец задачи 1
   */

  /*
   * Задача 2
   * поиск чисел, делящихся на девятнадцать или тринадцать
   */
  std::cout << "--- Задача 2 ---" << std::endl;
  std::cout << "Поиск чисел, делящихся на 19 или 13" << std::endl;

  std::cout << "Введите число элементов массива: ";
  int N;
  std::cin >> N; // чтение количества элементов массива
  std::cout << "Вводите элементы массива через пробел: ";
  std::vector<int> nums(N, 0);
  for (int i = 0; i < N; i++)
    std::cin >> nums[i];

  std::cout << "Исходный массив: ";
  printVector(nums);

  std::cout << "Числа, делящиеся на 19 или 13: ";
  // лямбда-выражение для вывода вектора, если элемент вектора делится на 19
или 13
  for_each(nums.begin(), nums.end(), [&](int val) {
    if ((val % 19 == 0) or (val % 13 == 0)) std::cout << val << " ";
  });
  std::cout << std::endl;
```

```
  // очистка вектора
  nums.clear();
  /*
   * Конец задачи 2
   */

  return 0;
}
```

```
--- Задача 1 ---
Возведение чисел в определённую степень (в частности, в квадрат и куб)
Исходный массив: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Возведите в квадрат каждое число: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Возведите в куб каждое число: [1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]

--- Задача 2 ---
Поиск чисел, делящихся на 19 или 13
Введите число элементов массива: 10
Вводите элементы массива через пробел: 19 65 57 39 152 639 121 44 90 190
Исходный массив: [19, 65, 57, 39, 152, 639, 121, 44, 90, 190]
Числа, делящиеся на 19 или 13: 19 65 57 39 152 190
```